Does a longer tool list hurt MCP tool selection?

5 vs 20 vs 50 tools in context on a 200-task MCPToolBench++ subset

AI465 Agentic AI | Research Project, Phase 2: implementation and evaluation

U23AI052 | Individual work | SVNIT Surat

Objective, and what changed from Phase 1

Approved objective

Effect of tool-count context (5 vs 20 vs 50 tools) on MCP tool-selection accuracy and failure composition, on a stratified 200-task MCPToolBench++ subset (search, map, finance).

Metrics: AST accuracy, tokens, calls and latency per task, plus a failure taxonomy.

Fills the paper's gaps: no cost, no variance, no failure analysis.

Declared deviations

- **D1 Single-step only.** Public data has no multi-step items for these domains, so no AST DAG.
- **D2 Deterministic scorer** instead of the GPT-4o judge; audited against my labels.
- **D3 Missing required parameter** counts as an error (stricter).

Design: the tool list is the only thing that changes

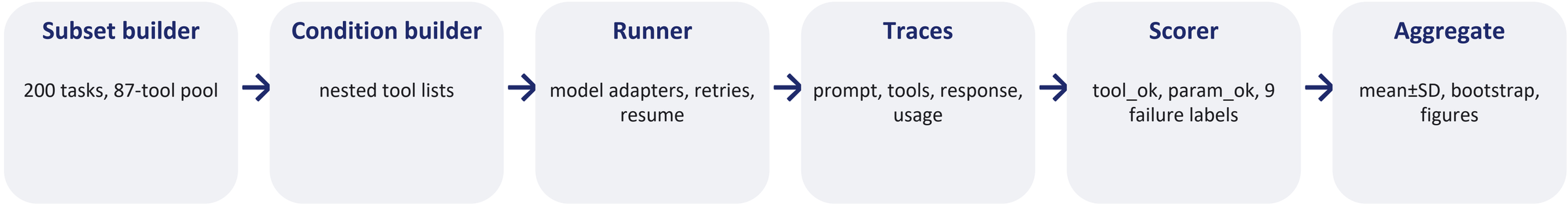


gold tool + up to 2 labelled similar tools are always present; distractors come from one seeded permutation per task

Condition	What differs	Isolates
c05 / c20 / c50	5, 20, 50 tools in context	effect of tool count (the objective)
c50_goldfirst / goldlast	gold tool placed first or last at 50 tools	position of the right tool
c05_padded	5 tools + neutral text to the 50-tool length	context length vs number of choices
c05_sc	5 tools, 8 samples, majority vote	matched token budget

Same system and user prompt files in every condition (a test asserts it). Every run: 200 tasks x 3 repetitions, temperature and seed pinned.

What I built

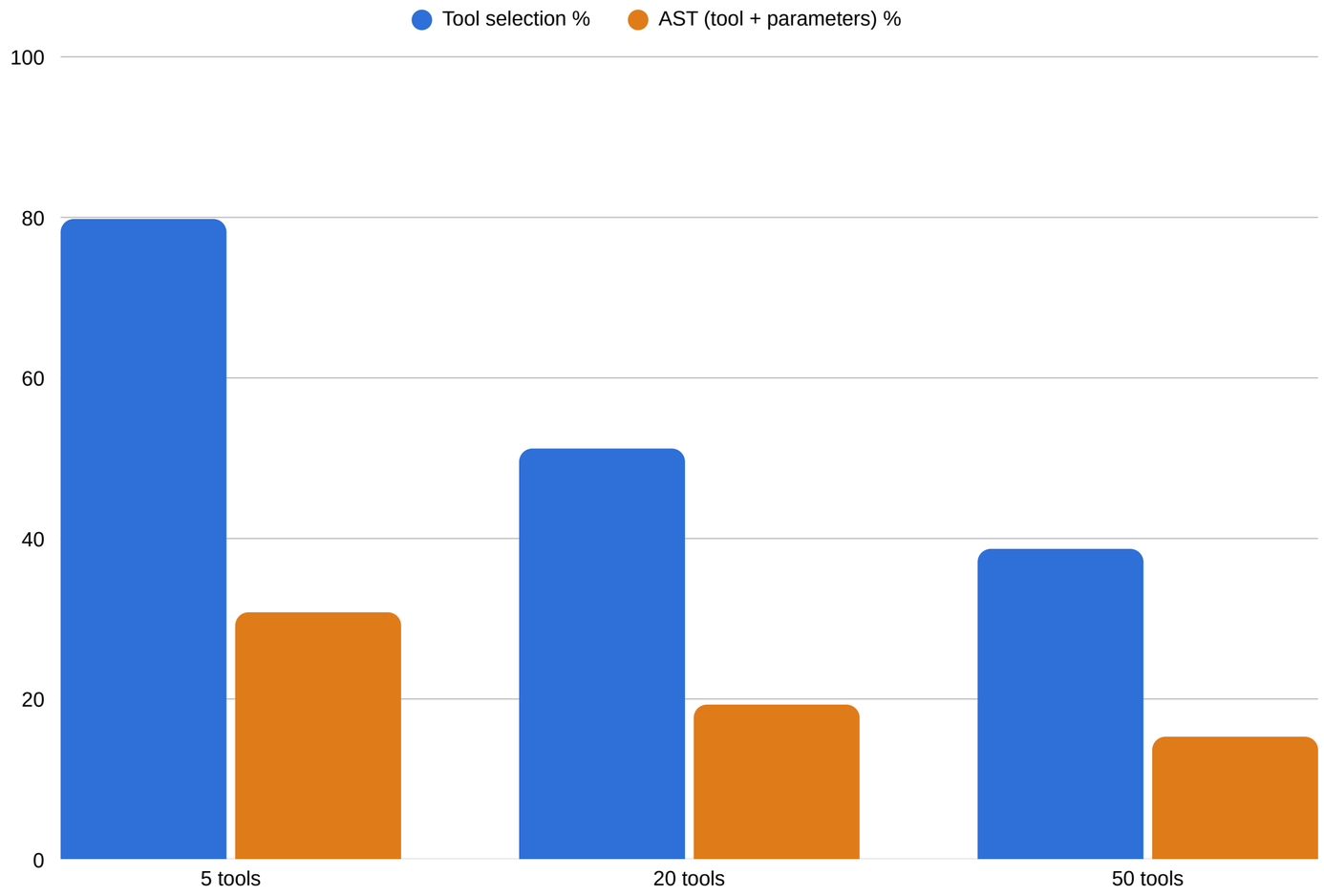


- **Own code:** runner, condition builder, scorer, statistics, report and slide generators, tests.
- **Borrowed:** the benchmark's task data and labels (commit 2bd35e5); its judge rules re-implemented, no code imported.
- **Robustness:** retries with backoff, rate limit, resume, jobs interleaved across conditions so model drift hits all of them.
- **Checked:** oracle test (200 gold labels score 100%), perturbation tests, 20-task audit against my own labels.

Experimental protocol

Requirement	How it is met
Repetition	3 full passes per condition; mean ± SD over runs; paired bootstrap over tasks
Pinned configuration	configs/config.yaml, manifest.json per run: model string asked and returned, temperature, top-p, seed, dates
Prompts as artefacts	prompts/*.txt; conditions differ only in the tool list (padding arm adds neutral text)
Metrics	tool and AST accuracy; model calls, input and output tokens, latency per task
Traces	every call: full prompt, tool list, raw response, usage, score, failure label
Judging	no LLM judge; deterministic scorer checked on 20 tasks against my labels
Reproducibility	one command per table; README states runtime and token cost

Results: accuracy vs number of tools

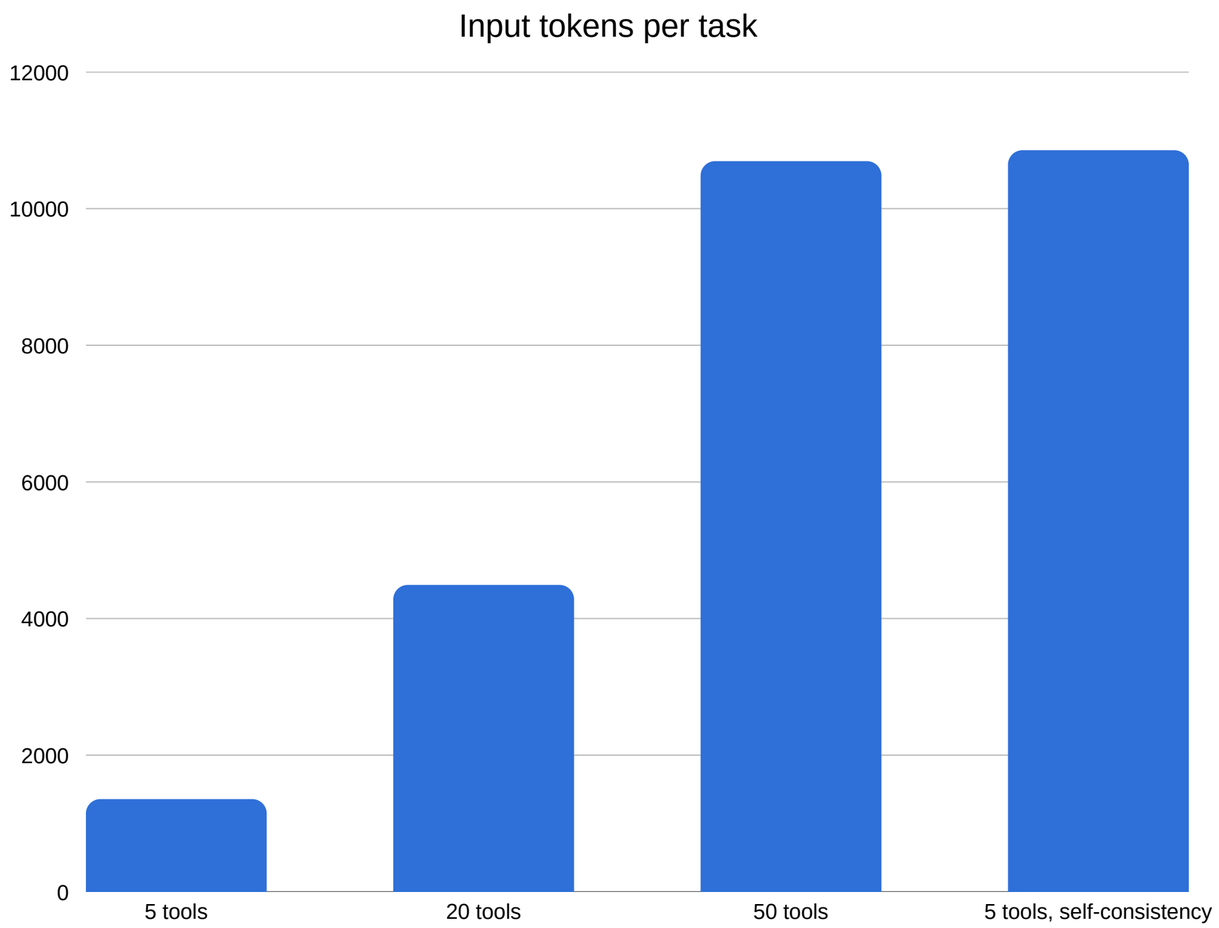


AST accuracy
30.8% -> 19.3% -> 15.3%
run-to-run SD 2.8 / 1.5 / 1.2 pp

5 to 50 tools, paired bootstrap
-15.5 pp
95% CI [-20.0, -11.2]
outside noise

Mean over 3 runs x 200 tasks.

Results: what the extra tools cost



Condition	AST %	Latency s	Tok / correct
5 tools	30.8	0.00	4517
20 tools	19.3	0.00	23421
50 tools	15.3	0.00	69975
5 tools, self-consistency	36.0	0.00	30965

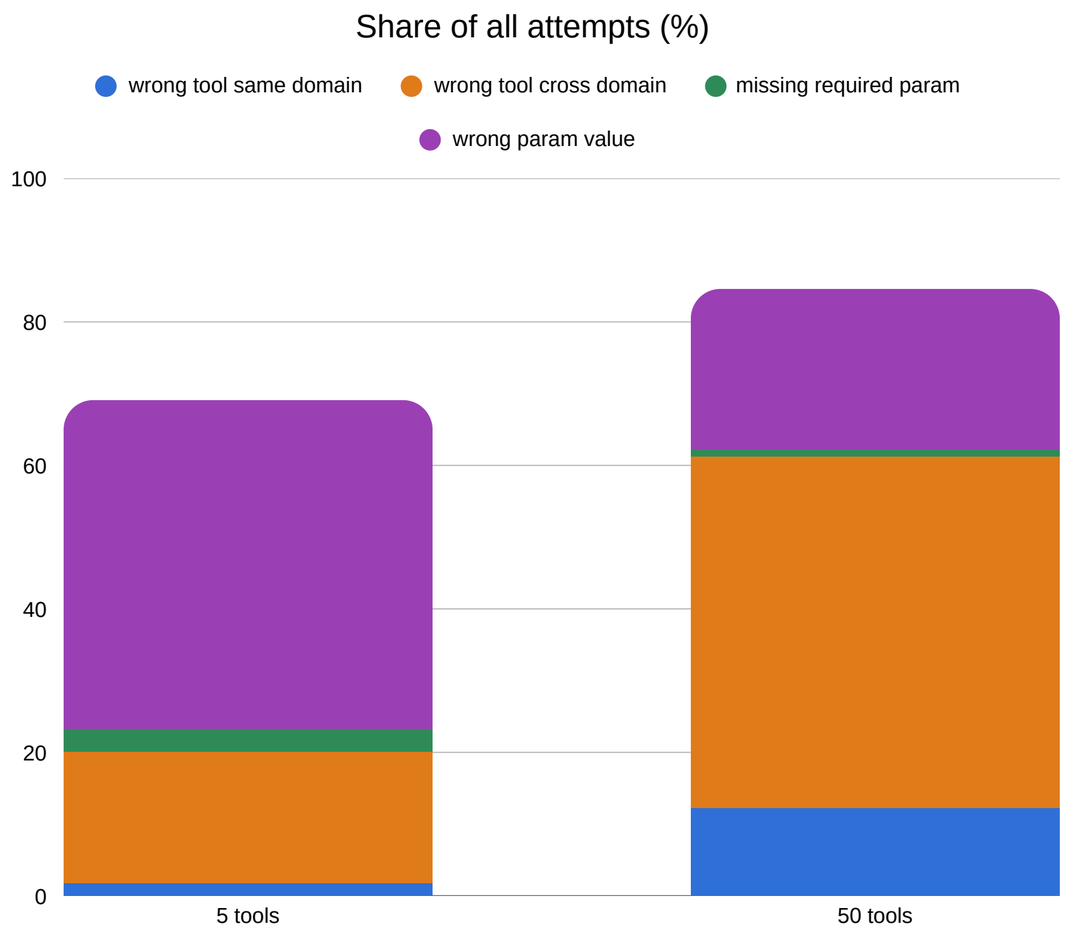
Calls per task: 1 in every condition except self-consistency (k votes).

Ablations: what explains the effect?

Comparison (AST accuracy)	Factor	Diff (pp)	95% CI (pp)	Beyond noise
50 tools: gold first vs shuffled	position	+0.3	[-2.5, +3.2]	no
50 tools: gold last vs shuffled	position	+0.5	[-2.5, +3.7]	no
5 tools padded to 50-tool length vs plain 5	context length	+0.0	[+0.0, +0.0]	no
5 tools, 8 votes vs plain 5	budget	+5.2	[+2.7, +7.7]	yes
50 tools vs 5 tools with 8 votes	matched budget	-20.7	[-26.0, -15.8]	yes

Differences are computed per task (mean over runs) and bootstrapped over the 200 tasks.

Why it fails: taxonomy and a traced case



Task 3ec01417 (search): solved in every 5-tool run, failed in every 50-tool run

Query: Can you extract the content from the URL https://www.wikipedia.org with basic depth and markdown format?
Gold: tavily-extract
Called: puppeteer_navigate -> wrong tool cross domain
Gold at position 15 of 50

Picked tool: Navigate to a URL
Gold tool: A powerful web content extraction tool that retrieves and processes raw content from specified URLs, ideal for data coll...

Cause: (state after reading the trace)